

TypeScript Cheatsheet

Introduction

TypeScript = JavaScript + static typing. Helps catch errors early and improve code quality.

Basic Types

string, number, boolean, null, undefined, void
any -> disables type checking (avoid)
unknown -> safer any, requires type checking
never -> function never returns (error/loop)

Example:

```
let name: string = "Harsha";  
let age: number = 24;  
function log(msg: string): void { console.log(msg); }  
function fail(): never { throw new Error("Error!"); }
```

Objects & Arrays

```
interface User { name: string; age?: number; }  
const u: User = { name: "Harsha" };
```

```
let arr: number[] = [1, 2, 3];  
let tuple: [string, number] = ["hi", 2];
```

Functions

```
function add(a: number, b: number = 5): number { return a + b; }  
type MathFn = (x: number, y: number) => number;
```

Union & Intersection

```
let val: string | number;  
type Admin = { role: string };  
type User = { name: string };  
type AdminUser = Admin & User;
```

Enums

```
enum Direction { Up, Down, Left, Right }  
let d: Direction = Direction.Up;
```

Type Narrowing

```
function printId(id: string | number) {  
  if (typeof id === "string") console.log(id.toUpperCase());  
  else console.log(id);  
}
```

Classes

```
class Animal {  
  constructor(public name: string) {}  
}
```

```
class Dog extends Animal {  
  bark() { console.log("Woof"); }  
}
```

Generics

```
function identity<T>(val: T): T { return val; }  
let num = identity<number>(5);
```

Utility Types

```
Partial<T> -> all optional  
Pick<T, K> -> pick fields  
Omit<T, K> -> remove fields  
Readonly<T> -> all readonly
```

Advanced Types

```
keyof -> keys of object  
typeof -> type of variable  
Indexed -> T[K]  
Conditional -> T extends U ? X : Y
```

Example (API)

```
async function fetchUser(id: number): Promise<User> {  
  const res = await fetch(`/api/user/${id}`);  
  return res.json();  
}
```